

# SILACOD.MA — PLATFORM SECURITY AUDIT

Codebase Vulnerability Analysis, Multi-Tenant Defense & Hardening Roadmap

## 1. Executive Summary & Audit Overview

A comprehensive static code analysis, architectural review, and access-control security audit was conducted on the **Silacod (Vegas) COD E-Commerce Platform** codebase. The assessment surveyed 52 backend route controllers, authentication middleware, database queries, third-party webhook integrations, and multi-tenant isolation boundaries. The review identified **10 key vulnerabilities and architectural weaknesses** that require remediation to protect the platform from cross-tenant data leaks, authentication bypasses, webhook spoofing, and unauthorized configuration changes.

| Finding ID | Vulnerability Area                                          | Severity | Target File(s)                  |
|------------|-------------------------------------------------------------|----------|---------------------------------|
| SEC-01     | JWT Token Type Confusion & 2FA Bypass                       | CRITICAL | middleware/auth.ts              |
| SEC-02     | Cross-Tenant IDOR on Orders & Shipping Labels               | HIGH     | routes/order.routes.ts          |
| SEC-03     | BOLA / IDOR on Lead Details & Modifications                 | HIGH     | routes/lead.routes.ts           |
| SEC-04     | Unauthenticated Webhook & Automated Stock Status Mutation   | HIGH     | routes/webhook.routes.ts        |
| SEC-05     | Cross-Tenant Real-Time Financial & Order Data Leak (SSE)    | HIGH     | routes/webhook.routes.ts        |
| SEC-06     | SSRF & OAuth Secret Exfiltration in Shopify Token Exchange  | HIGH     | routes/shopify.routes.ts        |
| SEC-07     | Unauthenticated WooCommerce Key Overwrite (Account Hijack)  | HIGH     | routes/woocommerce.routes.ts    |
| SEC-08     | Shell Command Execution in Media Transcoding (convertToMp3) | MEDIUM   | routes/upload.routes.ts         |
| SEC-09     | System User Password Injection in chpasswd Extension        | MEDIUM   | routes/admin.routes.ts          |
| SEC-10     | JWT Exposure via Query Parameters & Missing HSTS Headers    | LOW      | middleware/auth.ts, security.ts |

## 2. Detailed Technical Findings & Root Cause Analysis

### [SEC-01] Critical: JWT Token Type Confusion Allows 2FA & Password-Reset Bypass

**Affected Components:** backend/src/middleware/auth.ts, backend/src/routes/auth.routes.ts

**Vulnerability Description:** The authentication middleware `authenticate()` verifies incoming tokens with `jwt.verify(token, JWT_SECRET)` without checking the token's type claim. When an account has 2FA enabled, `/auth/login` returns a temporary 2FA token (`{ userId, type: '2fa' }`) meant solely for `/auth/login/2fa`. Because `authenticate()` does not reject non-access token types, an attacker with this temporary token can supply it in the `Authorization: Bearer` header to access all authenticated endpoints without supplying the required 2FA code.

**Remediation:** In `authenticate()`, strictly reject any token where `decoded.type && decoded.type !== 'access'` with HTTP 401.

```
// Fix in backend/src/middleware/auth.ts
const decoded = jwt.verify(token, process.env.JWT_SECRET!) as {
  userId: string; type?: string };
if (decoded.type && decoded.type !== 'access') {
  return res.status(401).json({ status: 'error', message: 'Invalid token type for authorization.' });
}
```

### [SEC-02] High: Insecure Direct Object Reference (IDOR) on Orders & Shipping Labels

**Affected Components:** backend/src/routes/order.routes.ts (Lines 221-235 & Lines 436-448)

**Vulnerability Description:** In `GET /api/v1/orders/:id`, the filter `where.vendorId = req.user.id` is only applied when `req.user.roleName === 'VENDOR'`. Non-vendor accounts (e.g. Influencers, Grossellers, unconfirmed accounts) bypass this check and can inspect any order by numeric ID, exposing recipient full names, delivery addresses, phone numbers, and

COD amounts. In addition, GET /api/v1/orders/parcel/:code/label allows any authenticated user to download any parcel label PDF without validating parcel ownership.

**Remediation:** Verify tenant ownership on GET /:id and GET /parcel/:code/label (ensure order.vendorId === req.user.id or an authorized helper/admin).

### [SEC-03] High: Broken Object Level Authorization (BOLA) on Lead Management

**Affected Components:** backend/src/routes/lead.routes.ts (Lines 2248-2305 & Lines 3428-3450)

**Vulnerability Description:** The lead detail endpoint GET /api/v1/leads/:id/detail checks call-center agent assignments, but does not verify vendor ownership. A vendor can inspect another vendor's lead by ID, exposing customer contact info, notes, and order history. In PATCH /api/v1/leads/:id, helper assignment scoping is not enforced, allowing helpers to modify leads across any vendor.

**Remediation:** Scope all lead queries and mutations with vendorId: req.user.id for vendors and verify helperUserAssignment for helpers.

### [SEC-04 & SEC-05] High: Unauthenticated Webhooks & Real-Time SSE Multi-Tenant Leak

**Affected Components:** backend/src/routes/webhook.routes.ts

**Vulnerability Description:**

1. **Unauthenticated Webhook:** The Coliaty webhook (POST /api/v1/webhook/coliaty) does not require a secret token, HMAC signature, or IP whitelist. Anyone on the internet can POST synthetic events, altering order statuses to DELIVERED or CANCELED and triggering automated stock adjustments.
2. **Cross-Tenant SSE Stream:** The real-time stream GET /api/v1/webhook/stream broadcasts global status\_update events—containing order numbers, prices, courier fees, and net profits—to all connected users without vendor filtering.

**Remediation:** Protect the webhook endpoint with a pre-shared secret token. Filter SSE stream events to send only updates matching data.vendorId === streamUser.id.

### [SEC-06 & SEC-07] High: Integration SSRF & OAuth Callback Hijacking

**Affected Components:** backend/src/routes/shopify.routes.ts, backend/src/routes/woocommerce.routes.ts

**Vulnerability Description:**

1. **Shopify OAuth SSRF:** In POST /api/v1/shopify/token, the domain parameter shop is accepted directly without regex validation. An attacker can specify an external host, causing the backend to POST the platform's SHOPIFY\_CLIENT\_SECRET to an arbitrary endpoint.
2. **WooCommerce Public Callback:** POST /api/v1/woocommerce/auth-callback is an unauthenticated public route that overwrites a vendor's WooCommerce API credentials without verifying a signed one-time state token.

**Remediation:** Validate that Shopify shop domains match /^[a-zA-Z0-9][a-zA-Z0-9\-\]\*\.myshopify\.com\$/ . Implement signed HMAC state verification for WooCommerce OAuth.

### [SEC-08, SEC-09, SEC-10] Medium & Low: Shell Execution, Password Injection & Protocol Hygiene

- **SEC-08 (Shell Execution):** In upload.routes.ts, convertToMp3 invokes FFmpeg via child\_process.exec(cmd) using a shell string. Replace with execFile('ffmpeg', args) to avoid shell interpolation.
- **SEC-09 (Password Injection):** In admin.routes.ts (Line 4825), the professional email tool pipes \${username}:\${password}\n into chpasswd without validating that the password contains no newlines or colons.
- **SEC-10 (Query Token & HSTS):** In middleware/auth.ts, accepting JWTs via req.query.token risks token exposure in server logs. Enforce headers/cookies and enable Strict-Transport-Security (HSTS) in production.

### 3. Remediation Action Plan & Hardening Milestones

| Phase                                        | Target Actions & Milestones                                                                                                    | Priority             | Estimated Effort |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------|------------------|
| Phase 1: Critical Hotfix                     | 1. Patch JWT verification to block non-access token types.<br>2. Implement token refresh logic and ownership check on refresh. | <b>P0 (Critical)</b> | 1-2 Hours        |
| Phase 2: Integration & Security Enhancements | 1. Secure API endpoints with OAuth2/OIDC.<br>2. Scope SSRF and time updates to mobile app.                                     | <b>P1 (High)</b>     | 2-3 Hours        |
| Phase 3: System & Hygiene Hardening          | 1. Migrate legacy TOTP from exec() to execFile().<br>2. Sanitize input against newline/color codes.                            | <b>P2 (Medium)</b>   | 1-2 Hours        |